

CSCI235 – Database Systems 2026 S2 Implementation Task 2 Due on 10 May 2026

Scope

The Implementation of Task 2 is related to the contents of topic on Indexing.

This Implementation is due by Sunday, 10 May 2026, 9:00 pm Singapore time. This task is worth 3% of the total assessment for the subject.

Only electronic submission through Moodle at: <https://moodle.uowplatform.edu.au/> is accepted. All email submission will be deleted and mark 0 ("zero") will be awarded.

For all the implemented tasks, your report or output must include a listing of all PL/pgSQL statements processed.

The submission procedure is explained at the end of this specification.

Objective

Two main objectives of this assignment are:

1. This assignment aims to help students understand how PostgreSQL uses indexes during query execution. You will learn how different types of index traversals occur (vertical vs horizontal) and how to distinguish between index scans and index-only scans based on query structure and column selection.
2. This assignment is designed to help students understand how PostgreSQL indexing improves query performance and enables index-only scans. Students will explore query plans, design efficient indexes, and analyse how PostgreSQL decides between index scan and index-only scan strategies.

Learning Objectives

- Understand the role and performance impact of indexes in PostgreSQL.
- Differentiate between sequential scan, index scan, and index-only scans.
- Identify suitable indexing strategies that minimize disk I/O and query execution time.
- Design indexes that make use of index-only scans for performance optimization.

Specification

If you have not done it yet, download the TPCHR sample database and load the sample TPCHR using user account **tpchr**. This will make the user account **tpchr** the owner of the TPCHR database.

Question 1 (1.0 mark)

Scenario

- a) Consider a relational database that consists of the relational tables created by the following CREATE TABLE statements:

```
CREATE TABLE PART(  
    P_PARTKEY      NUMBER(12)          NOT NULL,  
    P_NAME         VARCHAR(55)         NOT NULL,  
    P_RETAILPRICE  NUMBER(12,2)        NOT NULL,  
    CONSTRAINT PART_PKEY PRIMARY KEY (P_PARTKEY) );
```

```
CREATE TABLE SUPPLIER(  
    S_SUPPKEY      NUMBER(12)          NOT NULL,  
    S_NAME         CHAR(25)            NOT NULL,  
    S_NATIONKEY    NUMBER(12)          NOT NULL,  
    S_PHONE        CHAR(15)            NOT NULL,  
    CONSTRAINT SUPPLIER_PKEY PRIMARY KEY (S_SUPPKEY) );
```

```
CREATE TABLE PARTSUPP(  
    PS_PARTKEY     NUMBER(12)          NOT NULL,  
    PS_SUPPKEY     NUMBER(12)          NOT NULL,  
    PS_SUPPLYCOST  NUMBER(12,2)        NOT NULL,  
    CONSTRAINT PARTSUPP_PKEY PRIMARY KEY (PS_PARTKEY, PS_SUPPKEY),  
    CONSTRAINT PARTSUPP_FKEY1 FOREIGN KEY (PS_PARTKEY)  
        REFERENCES PART(P_PARTKEY),  
    CONSTRAINT PARTSUPP_FKEY2 FOREIGN KEY (PS_SUPPKEY)  
        REFERENCES SUPPLIER(S_SUPPKEY) );
```

For each query listed below, determine whether an index should be created to optimize its performance. Treat each query as an individual case.

- **If an index should be created:**
 - List the attributes that should form the index key.
 - Provide the CREATE INDEX statement to create the index.
 - Explain how the index will improve query performance. Note that the order of attributes in the index key is important.
- **If an index is not needed:**
 - Explain why query performance will not suffer without a new index.

Assume that all relational tables are large enough that full table scans are more time-consuming than indexed access.

- i.

```
SELECT *  
FROM PART  
WHERE P_NAME = 'GRAPHIC CARD '  
AND P_RETAILPRICE = 250;
```

 (0.2 mark)
- ii.

```
SELECT      PS_SUPPKEY  
FROM PARTSUPP  
WHERE PS_PARTKEY = 'PMB001';
```

 (0.2 mark)
- iii.

```
SELECT S_NAME  
FROM SUPPLIER;
```

 (0.2 mark)
- iv.

```
SELECT COUNT(*)  
FROM PART  
WHERE P_NAME = 'COOLING-FAN';
```

 (0.2 mark)
- v.

```
SELECT PS_SUPPKEY, COUNT(*)  
FROM PARTSUPP  
GROUP BY PS_SUPPKEY  
HAVING COUNT(PS_SUPPLYCOST) > 3;
```

 (0.2 mark)

Deliverables

Include all five SELECT statements in an SQL script file named IT2SolutionQ1.sql. For each query, include a report (IT2ReportQ1.pdf or IT2ReportQ1.txt or IT2ReportQ1.docx) explaining:

- Which part of the index is traversed (vertical / horizontal).
- Whether the query uses an index scan or index-only scan.
- Whether a **heap fetch (table access)** is required to satisfy the query, and provide the reasoning for your answer.

Question 2 (2.0 marks)

Using the relational table LINEITEM of the sample database TPCHR, for each one of the queries listed below:

- i. Find all the discount (I_discount) of all the items that are shipped (I_shipdate) most recently. Hint. Most recently mean the latest shipment date.
 - ii. Find the total number of items shipped by air (I_shipmode) in 1998 (I_shipdate).
 - iii. Find the order number (I_orderkey) and item number (I_linenum) that have the highest discount (I_discount).
 - iv. Find the total number of item per line status (I_linestatus). List the line status and the total items per line status.
 - v. Find the order key (I_orderkey), line item number (I_linenum), line status (I_linestatus), shipment date (I_shipdate) and shipment mode (I_shipmode) of all orders with the order number (I_orderkey) 1795718, 1799046, and 1794626.
- a) Construct an SQL statement that produces the required output specified in the statement. **(1.0 mark)**
- b) Find the smallest number of indexes that improve performance of a given collection of SELECT statements of a relational table LINEITEM. The smallest number of indexing means a database system will compute the five queries constructed in (a) using one or more indexes that you have created. Hint, you may create an index that can be used to compute more than one queries. Use the explain analyze and show plan statements to justify your solutions. **(1.0 mark)**

Please remember that **you must consider each one of the queries as an individual case!** Please remember that all relational tables are large enough to make full table scans more time consuming than accessing the tables through an index! It means that any solution in which an index is not used for query processing is incorrect.

Deliverables

Include all five SELECT statements in an SQL script file named IT2SolutionQ2.sql. For each query, include a report (IT2ReportQ2.pdf or IT2ReportQ2.txt or IT2ReportQ2.docx) containing the CREATE INDEX statements that improve the performance of the queries listed (i, ii, iii, iv, and v above) and the execution plan generated.

Submissions

This assignment is due by 9:00 pm (21:00 hours) **Sunday, 10 May 2026 at 9:00 pm Singapore time.**

Zip the files IT2SolutionQ1.sql, IT2SolutionQ2.sql, IT2ReportQ1.pdf, and IT2ReportQ2.pdf into IT2Solution.zip and submit the zipped file through Moodle in the following way:

- 1) Access Moodle at **<http://moodle.uowplatform.edu.au/>**
- 2) To login use a Login link located in the right upper corner the Web page or in the middle of the bottom of the Web page
- 3) When successfully logged in, select a site CSCI235 (SP226) Database Systems
- 4) Scroll down to a section Submissions of Implementation Tasks
- 5) Click at Submit your Implementation Task 1 here link.
- 6) Click at a button Add Submission
- 7) Move the .sql and .pdf (or .docx) or IT2Solution.zip (**or the zipped file**) into an area provided in Moodle. You can drag and drop files here to add them. You can also use a link *Add...*
- 8) Click at a button Save changes,
- 9) Click at check box to confirm authorship of a submission,
- 10) When you are satisfied, remember to click at a button Submit assignment.

A policy regarding late submissions is included in the subject outline.

Only one submission per student is accepted.

Implementation Task 2 is an individual assessed task, and it is expected that all its tasks will be solved individually without any cooperation with the other students. Plagiarism is treated seriously. Students involved will likely receive zero. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or over e-mail.

End of specification